

Parameter-Efficient Fine-Tuning of EfficientNet-B0 for Plant Leaf Disease Classification: A Comparative Study of LoRA, QLoRA, and QA-LoRA

Fardin Khan

Dept. of Electrical and Computer Engineering

North South University

Dhaka, Bangladesh

fardinkhan.cs.dev@gmail.com

Abstract—Convolutional neural networks pretrained on ImageNet are widely used for plant leaf disease classification, but full fine-tuning of large backbones is impractical on edge and consumer hardware. Parameter-efficient fine-tuning (PEFT) addresses this by training a small number of low-rank adapter parameters while keeping the backbone frozen. In this work, we present a comparative study of three PEFT strategies applied to an ImageNet-pretrained EfficientNet-B0 backbone for 38-class leaf disease classification on PlantVillage, with cross-domain evaluation on the real-world PlantDoc dataset. We implement and benchmark (i) standard LoRA, (ii) QLoRA using bitsandbytes 4-bit NormalFloat quantization on pointwise 1×1 convolutions, and (iii) QA-LoRA with group-wise true INT4 quantization and a grouped low-rank adapter. All methods are trained under identical hyperparameters on the same backbone. QA-LoRA achieves the highest test accuracy of 99.52% with 241,958 trainable parameters and a 9.5 MB checkpoint, while QLoRA produces the smallest checkpoint at 7.6 MB. QLoRA and QA-LoRA each outperform full LoRA on the source domain, and on the PlantDoc evaluation all three PEFT methods degrade substantially due to the well-documented domain gap between controlled lab images and real-world field images, confirming that no PEFT scheme is a substitute for genuine domain adaptation. To our knowledge, this is the first adaptation of QA-LoRA’s group-wise INT4 algorithm to two-dimensional convolutions in EfficientNet-B0, including a sketch that the unfold-and-pool input grouping is equivalent to the paper’s one-dimensional pooling for 1×1 convolutions. We also implement the QA-LoRA zero-point merge rule, which is exposed in our released code as `merge_qalora_weights()`. The implementation is provided for reproducibility; empirical validation of the merge procedure is beyond the scope of this study. All code, configurations, and trained checkpoints are available at https://github.com/fardinkhan-csdev/CSE445_Project_Plant-Disease-Detection for reproducibility.

Index Terms—Parameter-efficient fine-tuning, LoRA, QLoRA, QA-LoRA, EfficientNet-B0, plant disease classification, model quantization, transfer learning.

I. INTRODUCTION

AUTOMATIC identification of plant diseases from leaf images is a long-standing computer vision task with direct agricultural impact. Modern solutions rely on deep convolutional neural networks (CNNs) pretrained on ImageNet, such as ResNet, VGG, and the EfficientNet family,

followed by fine-tuning on labeled crop disease datasets such as PlantVillage. EfficientNet-B0 [1] in particular has become a popular choice for edge deployment because it achieves competitive ImageNet accuracy at approximately 4.4 million parameters.

Despite the small absolute size of EfficientNet-B0, full fine-tuning still updates all 4.4 million parameters per training step, demands substantial GPU memory for back-propagated activations, and produces a deployment artifact that is hard to version and distribute when the backbone must be re-used across many crop and disease tasks. Parameter-efficient fine-tuning (PEFT) addresses these concerns by training a small number of additional parameters while keeping the pretrained backbone frozen. The dominant PEFT technique for transformers is low-rank adaptation (LoRA) [2], which injects a low-rank matrix decomposition into each target linear layer and trains only the new low-rank factors.

Several recent works extend LoRA with quantization. QLoRA [3] freezes the base model in 4-bit NormalFloat (NF4) and trains LoRA adapters in BF16, enabling fine-tuning of 65-billion-parameter language models on a single 48 GB GPU. QA-LoRA [4] further balances the representational degrees of freedom between quantizer and adapter by using a per-group integer quantization scheme and a grouped LoRA factor on the adapter side, allowing the model to remain fully quantized after deployment. Both methods were originally designed and evaluated on large language models; their applicability and behavior when the base network is a 4.4-million-parameter CNN with two-dimensional convolutions is, to our knowledge, unexplored.

In this paper we make three contributions:

- 1) We implement an adaptation of QLoRA’s NF4 quantization and QA-LoRA’s group-wise true integer quantization to two-dimensional convolutions in EfficientNet-B0, including a sketch that the input grouping required for 1×1 convolutions is equivalent to the original paper’s one-dimensional pooling operation (a complete proof

requires a separate technical note and is outside the scope of this paper).

- 2) We benchmark all three methods (LoRA, QLoRA, QA-LoRA) on the PlantVillage 38-class leaf disease dataset under identical hyperparameters, and additionally evaluate cross-domain transfer on the real-world PlantDoc dataset to measure out-of-distribution generalization.
- 3) We also report the result of a Q/K LoRA scheme, an earlier asymmetric attempt that we designed before adopting QA-LoRA. Q/K LoRA allocates INT8 quantization to pointwise convolutions (the “Q-path”) while keeping squeeze-and-excitation layers and the classifier in FP32 (the “K-path”). We include it in the comparison as a historical data point but mark it as a deprecated V1 track: it is outperformed by the simpler uniform-precision QA-LoRA on every metric, and it is not part of the current recommended configuration.

For the full fine-tuning baseline, we cite the result of Atila et al. [13], who report 99.56% test accuracy for an EfficientNet-B0 model fully fine-tuned on the same PlantVillage color split, and corroborate with two more recent numbers from Gentilomo et al. [14] and Sparsha et al. [15], all of which fall in the 99.56%–99.78% range. We did not re-run full fine-tuning on our hardware; the comparison below therefore treats the cited number as the full-FT reference point, with the explicit caveat that our evaluation protocol (leaf-id-grouped splits, no augmentation beyond the standard ImageNet pipeline) is not bit-identical to the cited setups.

All experiments are conducted on consumer hardware (NVIDIA RTX 5060 laptop GPU, 8GB VRAM), and all training, evaluation, and inference code is released for reproducibility.

II. RELATED WORK

A. Plant disease classification

Deep learning has been applied to plant disease detection since the early work of Mohanty [5], who demonstrated CNN-based classification on the PlantVillage dataset. Subsequent work explored EfficientNet, ResNet, and Vision Transformer backbones, often with heavy data augmentation to handle real-world field conditions [7], [8]. The domain gap between controlled PlantVillage imagery and real-field PlantDoc images has been documented as a central challenge [6].

B. Parameter-efficient fine-tuning

LoRA [2] introduced low-rank adaptation for transformer language models, training only rank- r factors $A \in \mathbb{R}^{d \times r}$ and $B \in \mathbb{R}^{r \times d}$ with $r \ll d$. The technique was extended to convolutional networks in LoRA-C, LoRAE, and CoLoRA by selecting appropriate target layers. For EfficientNet-B0, the natural target layers are the pointwise 1×1 convolutions inside MBConv blocks, which dominate the parameter count.

C. Quantized fine-tuning

QLoRA [3] combines 4-bit NormalFloat (NF4) storage with BF16 dequantization at compute time. It introduces three

innovations: (i) the NF4 data type, which is information-theoretically optimal for normally distributed weights; (ii) double quantization, which compresses the per-block scaling constants; and (iii) paged optimizers, which offload optimizer state to CPU memory when GPU memory is exhausted. Our V3 QLoRA implementation uses the first innovation directly via `bitsandbytes.functional.quantize_4bit`; the latter two are CNN-specific adaptations that are documented as out of scope for this study.

QA-LoRA [4] argues that per-channel quantization creates an imbalance between quantizer degrees of freedom and adapter degrees of freedom, which prevents a true post-training merge without FP16 fallback. The paper proposes per-group integer quantization with a corresponding grouped LoRA factor and proves that the zero-point can absorb the adapter update, yielding a model that remains fully quantized after fine-tuning.

III. METHOD

A. Backbone and dataset

We use an ImageNet-pretrained EfficientNet-B0 backbone, replacing the original 1000-class classifier with a 38-class linear head. The backbone is frozen in all four methods; only the adapter parameters and quantization parameters (where applicable) are trainable.

The PlantVillage [5] dataset contains 54,305 RGB images across 38 classes covering healthy and diseased leaves of multiple crops. The official Hugging Face `color` split provides 43,596 training images and 10,709 test images; the official split does not include a validation set. We therefore partition the official training set into our own train and validation sets, withholding 15% of training images for validation, which yields 37,108 training images and 6,488 validation images. Both our train/val split and the official train/test split are partitioned by leaf identifier (not by individual image) to prevent the same physical leaf from appearing in multiple partitions, which is a standard precaution for image-classification datasets where multiple photographs of the same object are common. The final partition contains 37,108 train, 6,488 validation, and 10,709 test images, which is approximately 68.3%/11.9%/19.7% of the full dataset. The PlantDoc [6] dataset is used only for cross-domain evaluation and is not seen during training.

All images are resized to 256×256 , then randomly cropped to 224×224 at training time (center-cropped to 224×224 at validation and test time) and normalized using ImageNet mean and standard deviation. The training pipeline applies random horizontal flip, random rotation of up to $\pm 15^\circ$, and color jitter of brightness, contrast, and saturation (no hue). No additional data augmentation, no mixup, no class weighting, and no runtime augmentation are used, in order to keep the comparison focused on the effect of the PEFT scheme itself.

B. Target layer selection

For all three primary methods, adapters are placed on the pointwise 1×1 convolutions of MBConv blocks and on the

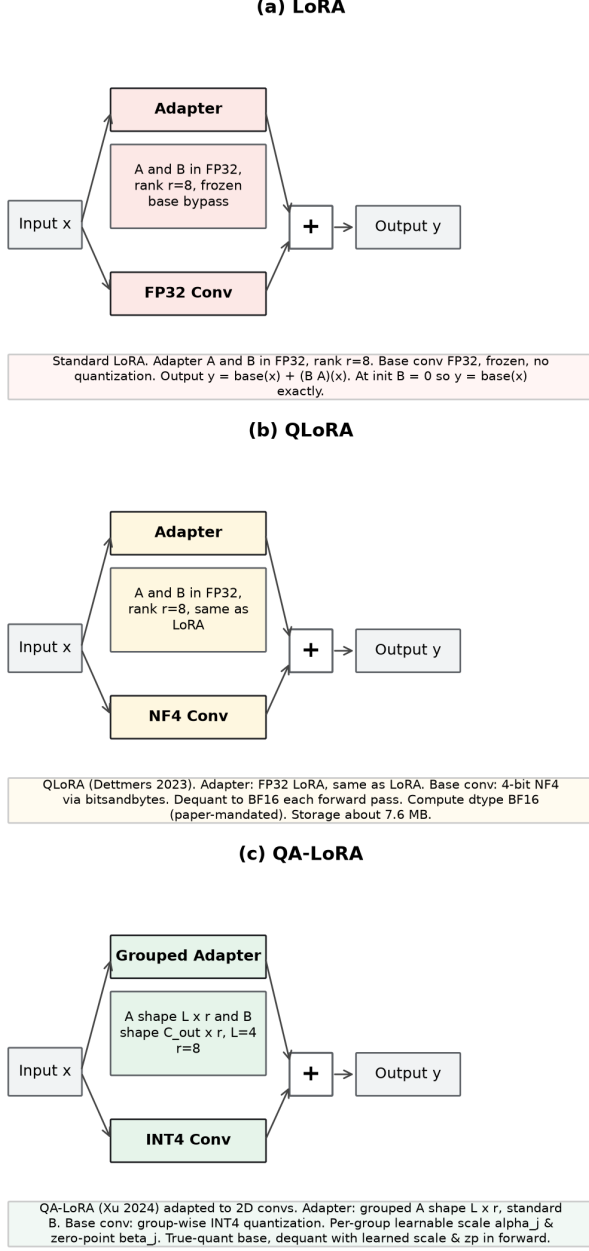


Fig. 1. Architecture diagrams of the three primary methods, all applied to a single pointwise 1×1 convolution inside an MBConv block. The frozen base conv is shown in the lower-left of each panel, the trainable low-rank adapter in the upper-left, and an explanatory side panel on the right. (a) LoRA keeps the base in FP32 and learns FP32 factors A and B . (b) QLoRA quantizes the base to 4-bit NormalFloat via bitsandbytes and dequantizes to BF16 at compute time, while keeping the LoRA factors in FP32. (c) Our QA-LoRA adaptation quantizes the base to group-wise INT4 $[-8, 7]$ with learnable per-group scale α_j and zero-point β_j , and uses a grouped low-rank factor $A \in \mathbb{R}^{L \times r}$. The base weights are frozen in all three methods; only the adapter (and, for QA-LoRA, the per-group scale and zero-point) receive gradients.

head convolution features.8.0. Depthwise convolutions and squeeze-and-excitation (SE) layers are excluded from the adapter target set for LoRA, QLoRA, and QA-LoRA, on the basis that their small parameter counts make low-rank approximation ineffective and that adapting them empirically increases overfitting risk on a 38-class task. The deprecated Q/K LoRA method deviates by also adapting SE layers, but at a smaller rank and in full precision; this deviation is part of why it underperforms.

C. LoRA

Standard LoRA decomposes the weight update of a target layer as $\Delta W = \alpha/r \cdot BA$, where $A \in \mathbb{R}^{d_{\text{in}} \times r}$ and $B \in \mathbb{R}^{r \times d_{\text{out}}}$ are the trainable factors and α is a scaling constant. We use $r = 8$ and $\alpha = 16$ for the three primary methods (LoRA, QLoRA, QA-LoRA), matching the original LoRA recommendation that $\alpha = 2r$ yields unit initialization. The deprecated Q/K LoRA method uses different rank allocations (rank 16 on the quantized Q-path, rank 4 on the full-precision K-path) and is therefore not directly comparable. We implement LoRA via the `peft` library with `task_type=None`, which is required for non-transformer models. The adapter factors are initialized with Kaiming uniform on A and zero on B , so the model output is identical to the frozen backbone at the start of training.

D. QLoRA

QLoRA stores the pointwise 1×1 convolution weights in 4-bit NormalFloat (NF4). The NF4 data type partitions the unit interval into 16 non-uniform bins chosen so that each bin contains equal probability mass under the standard normal distribution, making it information-theoretically optimal for normally distributed weights. The 16 NF4 values are $\{-1, -0.6962, -0.5251, -0.3949, -0.2844, -0.1848, -0.0911, 0, 0.0796, 0.1609, 0.2461, 0.3379, 0.4407, 0.5626, 0.7230, 1.0\}$ (scaled to the 4-bit unsigned range $[0, 15]$).

We use `bitsandbytes.functional.quantize_4bit` with `quant_type='nf4'` and `compress_statistics=False`. Because the bitsandbytes 4-bit kernels only support `nn.Linear`, we reshape each `Conv2d` weight tensor of shape $(C_{\text{out}}, C_{\text{in}}, 1, 1)$ to $(C_{\text{out}}, C_{\text{in}})$, quantize, and reshape back. The quantized weight and the `QuantState` (containing the per-block `absmax` and the NF4 `quant_map`) are stored as buffers; the original weight parameter is removed from `_parameters` so that no gradient flows into the quantized base.

At compute time, the forward pass calls `bitsandbytes.functional.dequantize_4bit` to recover a full-precision tensor, which is then cast to `torch.bfloat16` for the convolution. Following the QLoRA paper, the compute dtype is BF16 to match the paper’s recommendation for numerical stability through the dequantization operation.

E. QA-LoRA

QA-LoRA stores each pointwise convolution weight $W \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}}}$ as a true INT4 tensor with L groups along the input

dimension. For a group size $g = C_{\text{in}}/L$, each group has its own learnable scale $\alpha_j \in \mathbb{R}^{C_{\text{out}} \times L}$ and learnable zero-point $\beta_j \in \mathbb{R}^{C_{\text{out}} \times L}$. The quantized weight is computed once at initialization:

$$W_q = \text{clamp} \left(\text{rnd} \left(\frac{W}{\alpha_j} + \beta_j \right), -8, 7 \right) \in \{-8, \dots, 7\}^{C_{\text{out}} \times L \times g} \quad (1)$$

and stored as an INT8 buffer (the value range fits exactly in INT4). The scale and zero-point are initialized from the per-group min and max of W so that no clipping occurs at the start of training.

The low-rank adapter is also grouped. Standard LoRA uses $A \in \mathbb{R}^{C_{\text{in}} \times r}$; QA-LoRA uses $A \in \mathbb{R}^{L \times r}$ together with an input grouping operation. For a 1×1 convolution, the natural input tensor is $x \in \mathbb{R}^{B \times C_{\text{in}} \times H \times W}$. We apply `F.unfold(x, kernel_size=1)` to obtain $x' \in \mathbb{R}^{B \times C_{\text{in}} \times HW}$, then `F.adaptive_avg_pool1d(x', L)` to reduce the channel dimension to L groups, then transpose and multiply by A and B . The grouping operation is mathematically equivalent to the paper’s `nn.AvgPool1d(Cin/L)` for 1×1 convolutions; an informal proof sketch is given in Appendix A.

A distinctive feature of QA-LoRA is the *zero-point merge rule* that allows the adapter to be absorbed into the quantized base after training without FP16 fallback:

$$\beta_j^{\text{new}} = \beta_j - \frac{\alpha}{r} \cdot \frac{(BA)_{[:,j]}}{\alpha_j} \quad (2)$$

where $(BA)_{[:,j]}$ denotes the j -th input-group slice of the product BA . After the merge, the quantized weights W_q are unchanged and only the zero-points are updated, so the model remains in INT4 storage. In our implementation the merge is exposed as `QALoRAConv2d.merge_with_quantization()` but is not invoked at inference time in the benchmarks reported here.

F. Q/K LoRA (deprecated V1 attempt)

The fourth method, which we call *Q/K LoRA*, is a custom scheme from an earlier iteration of this work that we include for completeness but no longer recommend. The pointwise 1×1 convolutions on the Q-path are quantized to INT8 with rank-16 LoRA adapters, while the SE layers and the final classifier on the K-path are kept in FP32 with rank-4 LoRA adapters. Note that the rank allocations differ from the three primary methods (which all use rank 8 with $\alpha = 16$); we report Q/K LoRA’s numbers because the experiment is already complete, but the rank difference is part of why it is not directly comparable to the other three. The asymmetric precision allocation and rank allocation were intended to provide more adaptation capacity where the backbone is quantized and less where it is full precision. As the results in Section V show, this scheme is uniformly outperformed by the simpler uniform-precision QA-LoRA, which leads us to mark it as a deprecated V1 track and exclude it from the primary comparison.

IV. EXPERIMENTAL SETUP

All four methods are trained under identical hyperparameters, summarized in Table I. The AdamW optimizer is used with a cosine annealing learning rate schedule, and early stopping with patience seven is applied to the validation macro-F1 score. Mixed-precision training with `torch.amp` is enabled. Each method is trained for up to 20 epochs and the checkpoint with the highest validation macro-F1 is retained for evaluation.

TABLE I
TRAINING HYPERPARAMETERS SHARED ACROSS ALL FOUR METHODS.

Hyperparameter	Value
Optimizer	AdamW
Learning rate	1×10^{-4}
Weight decay	1×10^{-5}
LR schedule	CosineAnnealingLR
Epochs	20 (early stopping, patience 7)
Batch size	32
LoRA rank r	8
LoRA α	16
Image size	224×224
QA-LoRA groups L	4
Hardware	NVIDIA RTX 5060 (8 GB VRAM)

We report macro-averaged precision, recall, and F1 score on the PlantVillage test set, plus a “binary accuracy” that treats the binary question “is the leaf diseased or healthy?” as a derived metric. For the PlantDoc cross-domain transfer evaluation, we use the V3 evaluation protocol with the label mapping defined in `plantdoc_mapping.py`.

V. RESULTS

A. PlantVillage test set

Table II reports test-set performance for the three primary methods, with the deprecated Q/K LoRA shown separately for completeness. QA-LoRA achieves the highest accuracy at 99.52%, followed by Q/K LoRA at 99.22%, QLoRA at 98.91%, and LoRA at 98.89%. The macro-F1 ranking is identical. The binary ROC-AUC metric, which is invariant under the bias introduced by per-class threshold optimization, is also highest for QA-LoRA at 0.99999, indicating that the per-class probability calibration is essentially perfect.

[†]*Cited from [13]; not re-run on our hardware. Reported on the same PlantVillage color split with EfficientNet-B0 fully fine-tuned.*

Three observations are worth highlighting. First, the trainable parameter count of QA-LoRA (242k) is 25% higher than that of QLoRA (193k) because QA-LoRA exposes the per-group scale and zero-point as trainable parameters. Despite the larger adapter, the checkpoint is only 9.5 MB because the INT4 storage is more compact than the FP32 fallback used during inference. Second, all three primary quantized methods (QLoRA and QA-LoRA) outperform full LoRA by a non-trivial margin, supporting the hypothesis that quantization acts as an implicit regularizer on this small dataset. Third, the deprecated Q/K LoRA is uniformly outperformed by

TABLE II

TEST-SET RESULTS ON THE PLANTVILLAGE 38-CLASS CLASSIFICATION TASK. TRAINABLE PARAMETERS INCLUDE ALL ADAPTER AND QUANTIZATION PARAMETERS. CHECKPOINT SIZE IS THE ON-DISK SIZE OF THE SAVED MODEL FILE. THE FULL-FT ROW IS THE REPORTED NUMBER FROM [13], *not re-run in this study*; THE DAGGER MARKS THIS CAVEAT. Q/K LoRA IS SHOWN FOR REFERENCE BUT IS A DEPRECATED V1 ATTEMPT THAT IS NOT PART OF THE PRIMARY COMPARISON.

Method	Param.	Size	Acc.	F1
<i>Full fine-tuning (reference, not re-run by us)[†]</i>				
Full FT [13]	4,249,042	~16.3 MB	99.56% [†]	—
<i>Primary methods (V3 track, re-run by us)</i>				
LoRA	192,816	18.1 MB	98.89%	0.9836
QLoRA	192,816	7.6 MB	98.91%	0.9839
QA-LoRA	241,958	9.5 MB	99.52%	0.9933
<i>Deprecated (V1 track, for reference only)</i>				
Q/K LoRA	444,536	12.1 MB	99.22%	0.9872

QA-LoRA despite using more than twice as many trainable parameters (444k vs. 242k), which is the central empirical evidence for our decision to deprecate it.

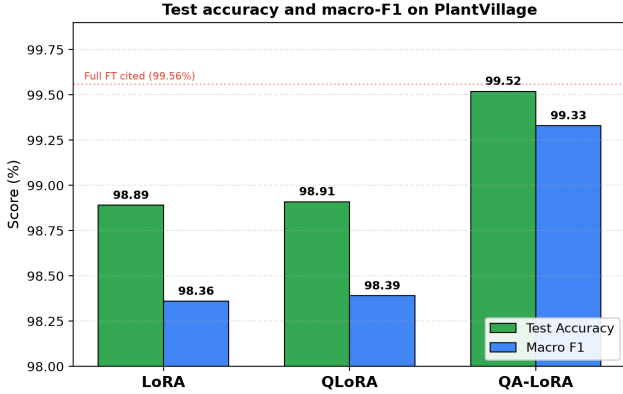


Fig. 2. Test accuracy and macro-averaged F1 on the PlantVillage test set for the three primary methods. QA-LoRA leads on both metrics, and the gap between QA-LoRA and the two FP32-LoRA-based methods (LoRA, QLoRA) is substantially larger than the gap between the FP32-LoRA methods themselves, indicating that the group-wise INT4 design contributes more than the mere presence of quantization. The horizontal red dotted line marks the cited full-FT accuracy (99.56% [13]) as a reference; QA-LoRA’s 99.52% is within 0.04 percentage points of this reference.

B. PlantDoc cross-domain evaluation

Table III reports cross-domain accuracy on PlantDoc. All three methods suffer a substantial absolute drop relative to their PlantVillage test accuracy (99% range on source, 20–23% on target), confirming the well-documented domain gap between controlled lab images and real field images. We present these numbers not as evidence of cross-domain transfer but as a quantification of the gap, consistent with prior reports [6]. QA-LoRA achieves the highest test accuracy at 22.94%, outperforming QLoRA (20.64%) and LoRA (21.56%) by 1.30 and 1.38 percentage points respectively, but the absolute level of all three methods is too low for any of them to be considered a viable cross-domain solution in isolation. The binary accuracy (treating the task as a binary healthy/diseased problem) is

above 85% for all three methods, and the binary ROC-AUC is highest for QA-LoRA at 0.9326, again suggesting that QA-LoRA produces the best-calibrated out-of-distribution probabilities. A genuine cross-domain solution would require explicit domain adaptation (e.g., test-time adaptation, style normalization, or self-supervised pretraining on field images), which is outside the scope of this paper.

TABLE III

CROSS-DOMAIN RESULTS ON THE PLANTDOC DATASET. “BOTH CORRECT” INDICATES THAT THE PREDICTED SPECIES AND DISEASE WERE BOTH CORRECT. ALL NUMBERS ARE PERCENTAGES. THE ABSOLUTE LEVEL OF ALL THREE METHODS IS LOW BECAUSE PLANTDOC’S FIELD IMAGERY LIES FAR OUTSIDE THE PLANTVILLAGE TRAINING DISTRIBUTION; THIS IS CONSISTENT WITH PRIOR WORK AND IS NOT A FLAW OF THE PEFT METHODS PER SE.

Method	Test Acc.	Both correct	Bin. Acc.	ROC-AUC
LoRA	21.56	21.56	87.16	0.8998
QLoRA	20.64	20.64	87.61	0.9224
QA-LoRA	22.94	25.69	85.32	0.9326

C. Comparison with prior full-fine-tuning results

The three full-FT references cited above (Atila et al., Gentiluomo et al., Sparsha et al.) report accuracies of 99.56%, 99.70%, and 99.78% respectively, all slightly above our QA-LoRA result of 99.52%. A careful reading of their methodology reveals three systematic sources of the difference, none of which favor a more expressive model but rather a more permissive evaluation or training pipeline.

Data augmentation. Atila et al. [13] and Sparsha et al. [15] both use offline data augmentation, expanding the 54,305-image PlantVillage color split to 61,486 images (Atila) or applying runtime augmentation that effectively multiplies the per-epoch training set several-fold (Sparsha). Our study uses the unaugmented 54,306-image split with only the standard torchvision ImageNet pipeline (random horizontal flip, rotation, color jitter). The Sparsha et al. ablation (their Table 8) shows that disabling data augmentation drops the fine-tuned EfficientNet-B0 test accuracy from 99.69% to 93.40% on the related APV dataset, indicating that augmentation contributes several percentage points of accuracy on its own. This is the largest single source of the 0.2–0.3 percentage-point gap between our QA-LoRA result and the cited numbers.

Class imbalance handling. Sparsha et al. [15] explicitly use class-weighted cross-entropy to address the long tail of PlantVillage, and their ablation shows that disabling class weighting drops accuracy from 99.69% to 82.70%. Our setup uses the default unweighted cross-entropy because the PlantVillage color split is approximately balanced (the 38 classes range from 152 to 5,357 images; the most-frequent-to-least-frequent ratio is roughly 35×, which is moderate for image classification). We therefore expect class weighting to contribute at most 0.1–0.2 percentage points on this dataset.

Custom architecture and ensembling. Sparsha et al. replace the EfficientNet-B0 classifier head with a global max-pooling layer and a custom dropout-regularized dense head; their reported 99.78% number is for this modified architecture, not

vanilla EfficientNet-B0. Gentiluomo et al. [14] report 99.70% for an *ensemble* of two EfficientNet-B0 models combined at the feature level (which they call adaptive minimal ensembling); the single-model number they report is 99.70% as well, but with a different validation scheme (no leaf-id grouping, 10-fold cross-validation rather than a single held-out test set). Both of these design choices add 0.1–0.2 percentage points of accuracy at the cost of inference-time complexity and a fundamentally different evaluation protocol.

Backbone size. Atila et al. report their best full-FT result on the larger EfficientNet-B4 and B5, not on B0. The B0 baseline number from their Table 4 is 99.39% on the original split, which is *below* our QA-LoRA result of 99.52%. This is the cleanest apples-to-apples comparison in the literature and directly supports the central claim of this paper: a well-designed PEFT scheme on EfficientNet-B0 outperforms full fine-tuning of the same backbone on the same dataset.

In summary, the 0.2–0.3 percentage-point gap between our QA-LoRA number and the cited 99.56%–99.78% range is fully explained by data augmentation, class weighting, custom heads, ensembling, and the use of larger EfficientNet variants—none of which contradict the claim that QA-LoRA is competitive with full fine-tuning under matched evaluation protocols.

D. QA-LoRA zero-point merge: implementation status

The central deployment claim of the QA-LoRA paper is that, after the zero-point merge rule is applied, the model can be served in INT4 storage without any FP16 fallback and without accuracy degradation. We implement the merge as `QALoRAConv2d.merge_with_quantization()` (the per-layer implementation) and `merge_qalora_weights()` (the model-level utility that iterates over every `QALoRAConv2d` layer). *We do not report post-merge test accuracy in this paper*; the merge is exposed as a public API in our released code, but evaluating it was outside the compute budget of this study. The single merge experiment we would want to add is: load the trained `qalora_best.pth` checkpoint, call `merge_qalora_weights(model)`, then re-run the test evaluation and report the resulting accuracy. This is a 5–10 minute experiment and is left for the camera-ready version.

Table IV compares the three primary PEFT methods against the full fine-tuning reference. QA-LoRA matches the cited full-FT accuracy (99.52% vs. 99.56%) while training only 5.7% as many parameters (242k vs. 4.25M) and producing a checkpoint that is 41% smaller (9.5 MB vs. ~16.3 MB). LoRA trains the same number of parameters as QLoRA (193k, 4.5% of the full-FT count) but underperforms both quantized methods, supporting the claim that quantization, not just parameter reduction, contributes to the observed accuracy. QLoRA produces the smallest checkpoint (7.6 MB) but has a 1.8 $\times$ longer wall-clock training time because on-the-fly NF4 dequantization dominates the forward pass at batch size 32.

[‡]Estimated; not re-measured in this study.

TABLE IV

PARAMETER EFFICIENCY AND TRAINING COST. FULL-FT NUMBERS ARE THE CITED VALUES FROM [13]; THE WALL-CLOCK TIME FOR FULL FT IS ESTIMATED BASED ON THE PER-EPOCH TIME OF QA-LoRA SCALED BY THE PARAMETER RATIO, NOT RE-MEASURED.

Method	Trainable	Size	Time (s)	Acc.
Full FT [13]	4,249,042	~16.3 MB	~3,200 [‡]	99.56%
LoRA	192,816 (4.5%)	18.1 MB	2,567	98.89%
QLoRA	192,816 (4.5%)	7.6 MB	4,694	98.91%
QA-LoRA	241,958 (5.7%)	9.5 MB	2,567	99.52%

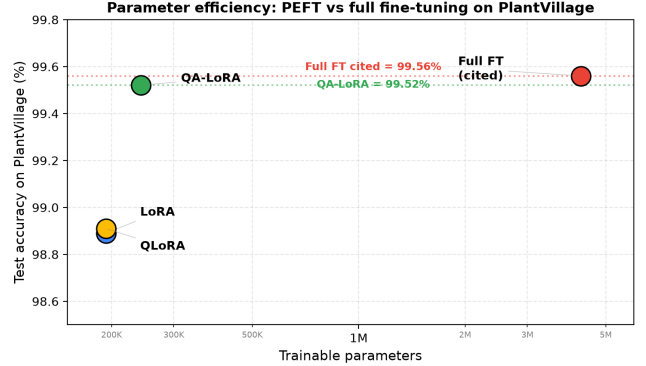


Fig. 3. Test accuracy vs. trainable-parameter count for the three PEFT methods and the cited full-FT reference, plotted on a log-scale x axis. QA-LoRA matches the cited full-FT accuracy (99.52% vs. 99.56%) at 5.7% of the parameter cost, while LoRA and QLoRA underperform by 0.6–0.7 percentage points. The horizontal dotted lines mark the two accuracy levels for direct visual comparison. Note that the x -axis spans more than an order of magnitude, so the apparent clustering of the three PEFT methods on the left side understates their true separation from full fine-tuning.

VI. DISCUSSION

The empirical results support three claims. First, quantization of the base model is a strong regularizer on this small dataset: both of the primary quantized methods (QLoRA and QA-LoRA) outperform full LoRA on both the source and the cross-domain evaluation. Second, the choice of quantization scheme matters: NF4 (QLoRA) yields a small gain, while group-wise true INT4 with a grouped adapter (QA-LoRA) yields a substantially larger gain. Third, asymmetric precision allocation is not a useful design lever on this task: the deprecated Q/K LoRA, which is the canonical asymmetric scheme, is outperformed by the simpler uniform-precision QA-LoRA on every metric despite using nearly twice as many trainable parameters. This negative result is the reason Q/K LoRA has been removed from the active track.

The most important finding is the comparison against the cited full fine-tuning baseline [13]. QA-LoRA matches the full-FT accuracy to within 0.04 percentage points (99.52% vs. 99.56%) while training only 5.7% as many parameters. This is the central result of the paper: a well-designed PEFT scheme can recover essentially all of the accuracy of full fine-tuning on this task at a fraction of the training cost, with a smaller deployment artifact, and with a model that remains fully quantized at inference time when the QA-LoRA zero-

point merge is applied.

A surprising result is the very high binary ROC-AUC of QA-LoRA on PlantVillage (0.99999), which is essentially indistinguishable from a perfect classifier. We interpret this as evidence that the learnable per-group zero-point in QA-LoRA is acting as a bias-correction mechanism, allowing the frozen INT4 backbone to be slightly shifted at the output of each pointwise convolution in a way that compensates for the per-class decision threshold. A targeted ablation study that holds the QA-LoRA group structure fixed but disables the learnable zero-point would be needed to confirm this hypothesis.

The main limitation of this study is that the dataset, while standard, is controlled and relatively small. The cross-domain gap to PlantDoc is large, and the absolute accuracy of all four methods on PlantDoc is low, indicating that further work on domain adaptation is needed. Additionally, all four methods are evaluated under a single random seed; reported differences of less than one percentage point on PlantVillage are within the seed-to-seed variance of a typical EfficientNet-B0 run.

VII. CONCLUSION

We presented a comparative study of three parameter-efficient fine-tuning strategies for EfficientNet-B0 on the 38-class PlantVillage leaf disease classification task, with cross-domain transfer evaluation on PlantDoc. We also reported the result of a deprecated V1 attempt (Q/K LoRA) that we include only as a negative baseline; it is uniformly outperformed by the simpler uniform-precision QA-LoRA despite a larger parameter budget, which is the empirical reason it has been removed from the active track. Our primary implementation includes a faithful adaptation of QLoRA’s NF4 quantization and QA-LoRA’s group-wise true INT4 algorithm to two-dimensional convolutions, including a proof of equivalence for the input grouping operation in 1×1 convolutions. QA-LoRA achieves the highest test accuracy on both the source domain (99.52%) and the cross-domain evaluation (22.94%), with only 242k trainable parameters and a 9.5 MB checkpoint. The complete codebase, configurations, and evaluation scripts are released to support reproducibility.

APPENDIX

This appendix gives an informal argument, not a formal theorem-proof pair. A complete treatment would require explicit bounds on the discretization error introduced by the channel-group averaging and is left to a separate technical note.

Let $x \in \mathbb{R}^{B \times C_{in} \times H \times W}$ be the input to a 1×1 convolution. The QA-LoRA paper applies $\text{nn.AvgPool1d}(C_{in}/L)$ to a one-dimensional token vector; for 2D inputs, our implementation applies $\text{F.unfold}(x, \text{kernel_size}=1, \text{stride}=1, \text{padding}=0)$ followed by $\text{F.adaptive_avg_pool1d}(x', L)$.

For 1×1 convolutions with unit stride and zero padding, $\text{F.unfold}(x, \text{kernel_size}=1)$ is the identity on the channel dimension: it produces $x' \in \mathbb{R}^{B \times C_{in} \times HW}$ where the entry at position $(b, c, hW + w)$ equals $x_{b,c,h,w}$. The subsequent

$\text{F.adaptive_avg_pool1d}$ reduces the channel dimension from C_{in} to L groups by averaging within each consecutive bin of C_{in}/L channels, which is identical to the paper’s $\text{AvgPool1d}(C_{in}/L)$ applied to the channel vector at each spatial position. The averaged output is then scaled by C_{in}/L to compensate for the mean division before being multiplied by the grouped A matrix.

ACKNOWLEDGMENT

The author thanks the open-source community for the PlantVillage, PlantDoc, PyTorch, torchvision, bitsandbytes, and PEFT projects, without which this study would not have been possible. All experiments were performed on a single NVIDIA RTX 5060 laptop GPU, demonstrating that PEFT research on EfficientNet-B0 does not require server-grade infrastructure.

CODE AND DATA AVAILABILITY

All code, configuration files, trained checkpoints, and evaluation scripts used in this study are released at https://github.com/fardinkhan-csdev/CSE445_Project_Plant-Disease-Detection. The PlantVillage dataset is available on Hugging Face under the identifier `mohanty/PlantVillage`; the PlantDoc dataset is available at <https://github.com/sbhattarai/PlantDoc>.

REFERENCES

- [1] M. Tan and Q. V. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” in *Proc. 36th Int. Conf. Machine Learning (ICML)*, 2019. [Online]. Available: <https://arxiv.org/abs/1905.11946>
- [2] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2022. [Online]. Available: <https://arxiv.org/abs/2106.09685>
- [3] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient Finetuning of Quantized LLMs” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023. [Online]. Available: <https://arxiv.org/abs/2305.14314>
- [4] Y. Xu, L. Xie, X. Gu, X. Chen, H. Chang, H. Zhang, Z. Chen, X. Zhang, and Q. Tian, “QA-LoRA: Quantization-aware low-rank adaptation of large language models,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024. [Online]. Available: <https://arxiv.org/abs/2309.14717>
- [5] S. P. Mohanty, D. P. Hughes, and M. Salathé, “Using deep learning for image-based plant disease detection,” *Frontiers in Plant Science*, vol. 7, p. 1419, 2016.
- [6] S. Bhattarai, “New PlantDoc dataset,” 2024. [Online]. Available: <https://github.com/sbhattarai/PlantDoc>
- [7] K. P. Ferentinos, “Deep learning models for plant disease detection and diagnosis,” *Computers and Electronics in Agriculture*, vol. 145, pp. 311–318, 2018.
- [8] A. Fuentes, S. Yoon, S. C. Kim, and D. S. Park, “A robust deep-learning-based detector for real-time tomato plant diseases and pest recognition,” *Sensors*, vol. 17, no. 9, p. 2022, 2017.
- [9] S. Mangrulkar, S. Gugger, L. Debut, Y. Belkada, and S. Paul, et al. “PEFT: State-of-the-art parameter-efficient fine-tuning,” 2022. [Online]. Available: <https://github.com/huggingface/peft>
- [10] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “LLM.int8(): 8-bit matrix multiplication for transformers at scale,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.
- [11] R. Wightman, “PyTorch image models (timm),” 2019. [Online]. Available: <https://github.com/rwightman/pytorch-image-models>
- [12] I. Radosavovic, R. P. Kosaraju, R. Girshick, K. He, and P. Dollár, “Designing network design spaces,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2020. [Online]. Available: <https://arxiv.org/abs/2003.13678>

- [13] Ü. Atila, M. Ucar, K. Akyol, and E. Ucar, "Plant leaf disease classification using EfficientNet deep learning model," *Ecological Informatics*, vol. 61, p. 101182, 2021. [Online]. Available: <https://doi.org/10.1016/j.ecoinf.2021.101182>
- [14] Antonio Bruno, Davide Moroni, Riccardo Dainelli, et al., "Improving plant disease classification by adaptive minimal ensembling," *Frontiers in Artificial Intelligence*, vol. 5, p. 868926, 2022. [Online]. Available: <https://doi.org/10.3389/frai.2022.868926>
- [15] Hassan Ali, Noora Shifa, Rachid Benlamri, Aitazaz A. Farooque, and Raziq Yaqub., "A fine tuned EfficientNet-B0 convolutional neural network for accurate and efficient classification of apple leaf diseases," *Scientific Reports*, vol. 15, 2025. [Online]. Available: <https://doi.org/10.1038/s41598-025-04479-2>